

תכונות זמן לינארי

Linear-Time Properties

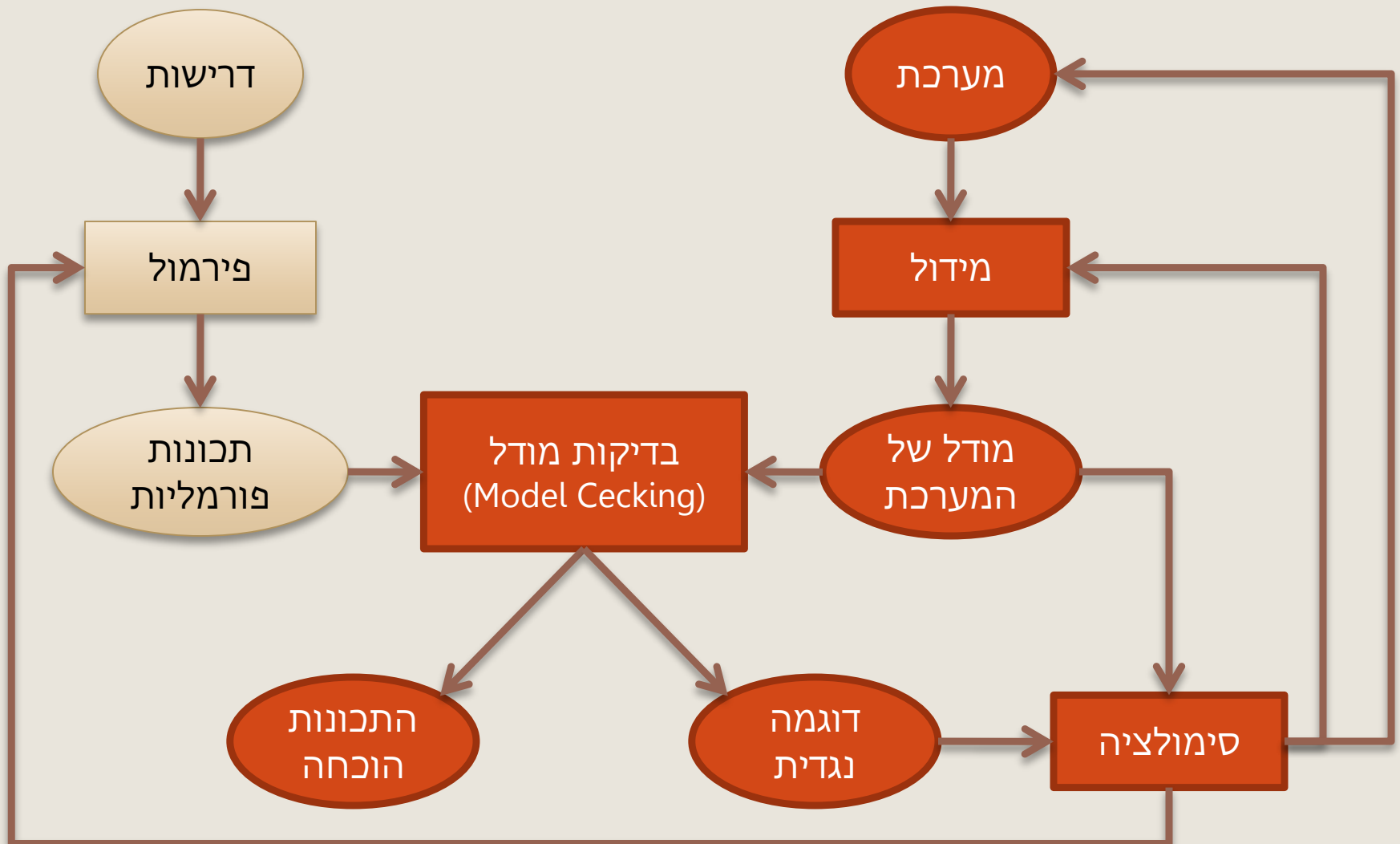
גרא וייס

המחלקה למדעי המחשב

אוניברסיטת בן-גוריון



בדיקות מודל (Model Checking)





תוכנית לפרק השני

נַעֲמַת מערכות מעברים מול תכונות רצויות ושאינן רצויות

קיימים סוגים שונים של תכונות:

- אנחנו נתמקד בתכונות של **הריצות** של התוכנית (להבדיל מתכונות של התוכנית עצמה)
- לתכונות כאלה נקרא "**תכונות זמן לינארי**" – **Linear Time Properties**

נעבור מתכונות פשוטות לתכונות מורכבות יותר:

- **קִפְאוֹן** – המערכת אף פעם לא "נתקעת"
- **תכונות "שְׁמוּרָה"** – בכל ריצה, בכל מצב בריצה, מתקיים תנאי רצוי
- **תכונות בטיחות** – לא יכול להיות שהמערכת תבצע רצף נתון של פעולות לא חוקית
- **תכונות מורכבות יותר (הכוללות חיות)** – למשל, פסוק אטומי מסוים מתקיים באופן תדיר

נגדיר את סוגי התכונות השונים ונִפְתַּח אלגוריתמים לבדיקה אם מערכת מעברים נתונה מקיימת תכונות נתונות

תוכן

קפאון



תכונות זמן ליניארי (Linear Time Properties)



שקילות עקבות (trace equivalence) ותכונות זמן ליניארי

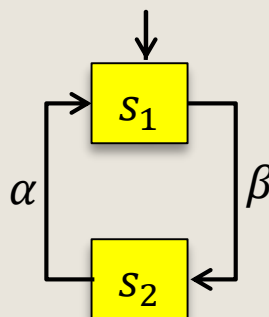
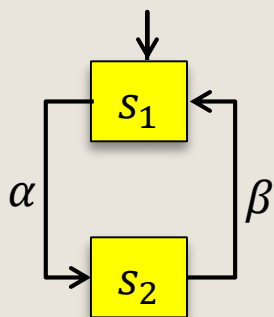


שמורות (invariants)



קפאון (deadlock)

- לתוכניות סדרתיות יש נקודת התחלה ונקודת סוף
- במערכת תגובתיות, בדרך כלל, לא רוצים שהחישוב יסתיים
- הגדרת המושג קפאון מבחינתנו: אם יש מצב נגיש וסופני (ללא עוקבים)
- מצב נפוש: שתי מערכות מעברים החוסמות זו את זו





דוגמה

$$\frac{M_{i,j}=\blacksquare \wedge M_{i,j+1}=\square}{M \xrightarrow{R(i,j)} M[\langle i,j \rangle := \square][\langle i,j+1 \rangle := \blacksquare]}$$

$$\frac{M_{i,j}=\blacksquare \wedge M_{i,j+1}=\boxtimes \wedge M_{i,j+2}=\square}{M \xrightarrow{R(i,j)} M[\langle i,j \rangle := \square][\langle i,j+1 \rangle := \blacksquare][\langle i,j+2 \rangle := \boxtimes]}$$

$$\frac{M_{i,j}=\blacksquare \wedge M_{i,j-1}=\square}{M \xrightarrow{L(i,j)} M[\langle i,j \rangle := \square][\langle i,j-1 \rangle := \blacksquare]}$$

$$\frac{M_{i,j}=\blacksquare \wedge M_{i,j-1}=\boxtimes \wedge M_{i,j-2}=\square}{M \xrightarrow{L(i,j)} M[\langle i,j \rangle := \square][\langle i,j-1 \rangle := \blacksquare][\langle i,j-2 \rangle := \boxtimes]}$$

$$\frac{M_{i,j}=\blacksquare \wedge M_{i+1,j}=\square}{M \xrightarrow{D(i,j)} M[\langle i,j \rangle := \square][\langle i+1,j \rangle := \blacksquare]}$$

$$\frac{M_{i,j}=\blacksquare \wedge M_{i+1,j}=\boxtimes \wedge M_{i+2,j}=\square}{M \xrightarrow{D(i,j)} M[\langle i,j \rangle := \square][\langle i+1,j \rangle := \blacksquare][\langle i+2,j \rangle := \boxtimes]}$$

$$\frac{M_{i,j}=\blacksquare \wedge M_{i-1,j}=\square}{M \xrightarrow{U(i,j)} M[\langle i,j \rangle := \square][\langle i-1,j \rangle := \blacksquare]}$$

$$\frac{M_{i,j}=\blacksquare \wedge M_{i-1,j}=\boxtimes \wedge M_{i-2,j}=\square}{M \xrightarrow{U(i,j)} M[\langle i,j \rangle := \square][\langle i-1,j \rangle := \blacksquare][\langle i-2,j \rangle := \boxtimes]}$$

$$M = \begin{pmatrix} \blacksquare & \blacksquare & \blacksquare & \blacksquare & \blacksquare & \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \square & \square & \square & \blacksquare & \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \boxtimes & \boxtimes & \square & \square & \square & \boxtimes & \blacksquare \\ \blacksquare & \square & \square & \square & \square & \square & \square & \blacksquare \\ \blacksquare & \square & \square & \square & \square & \square & \square & \blacksquare \\ \blacksquare & \square & \square & \boxtimes & \square & \square & \square & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare & \blacksquare & \blacksquare & \blacksquare & \blacksquare & \blacksquare \end{pmatrix}$$

האם ניתן להגיע ל-deadlock במערכת הזאת?



דוגמה: תקלה שקרתה ל-pathfinder במאדים

```
active proctype high() /* can run at any time */
{
end:    do
    :: h_state = waiting;
        atomic { mutex == free -> mutex = busy };
        h_state = running;

        /* critical section - consume data */

        atomic { h_state = idle; mutex = free }
    od
}
```

```
active proctype low()
provided (h_state == idle) /* scheduling rule */
{
end:    do
    :: l_state = waiting;
        atomic { mutex == free -> mutex = busy };
        l_state = running;

        /* critical section - produce data */

        atomic { l_state = idle; mutex = free }
    od
}
```

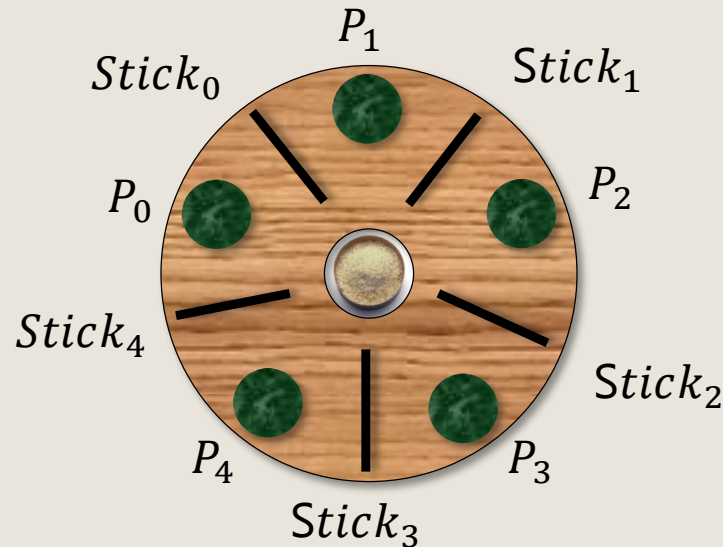
A high-priority process consumes data produced by a low-priority process. Data consumption and production happen under the protection of a mutex lock that conflicts with the scheduling priorities which can deadlock the system if high() starts up while low() has the lock.

There are 12 reachable states in the full state space - two of which are deadlock



דוגמה: המיסטיקנים הסינים

Dining Philosophers



פילוסופים סינים יושבים סביב שולחן שבמרכזו סיר אורז

חיי הסובבים מורכבים מאכילה, חשיבה והמתנה...

כדי לאכול אורז, פילוסוף זקוק לשני מקלות האכילה

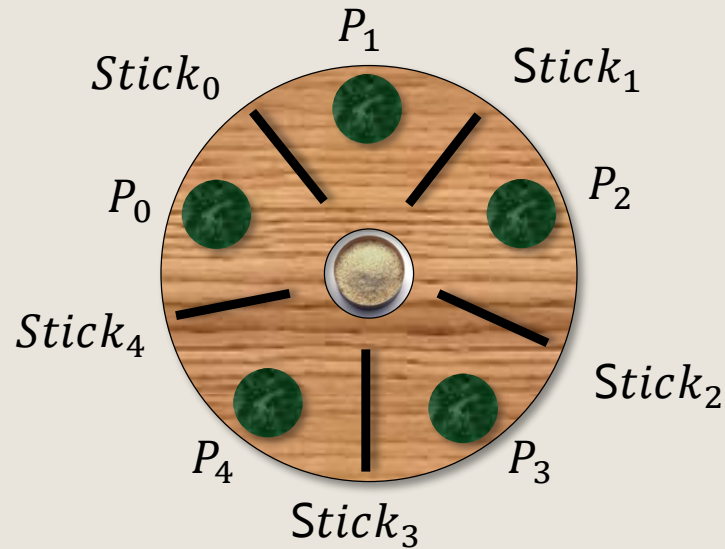
בין שני פילוסופים יש מקל אכילה אחד בלבד

רק אחד יכול להשתמש במקל בזמן נתון ואסור לאכול בידיים



דוגמה: המיסטיקנים הסינים

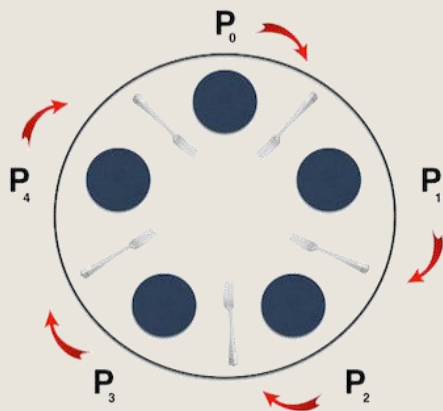
Dining (Chinese) Philosophers



לאור הדברים הפשוטים באמת
אנחנו חיים את חיינו
למשל בלי הסברים רק לקבל ולתת
זה לא קל אבל מה יש עוד בינינו?



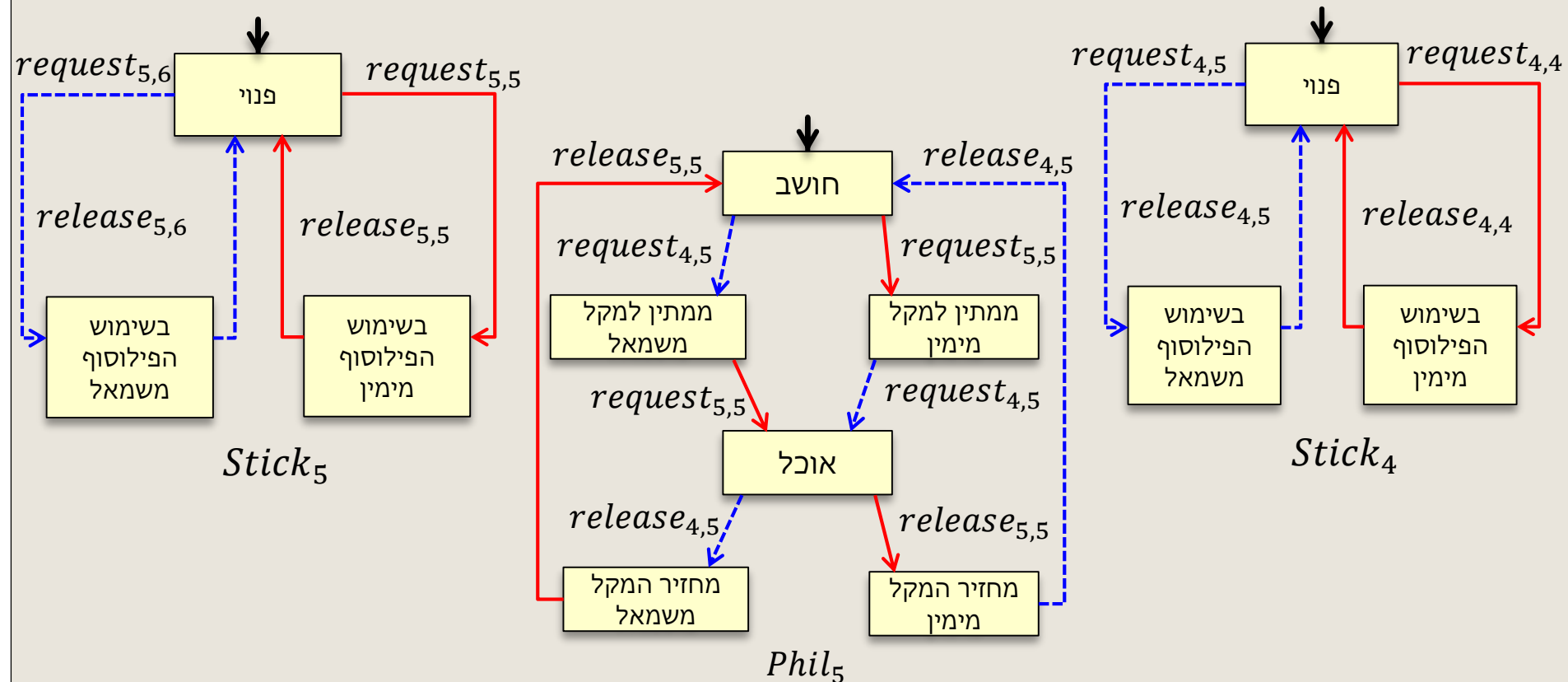
דוגמה: המיסטיקנים הסינים



- מצב קפאון: כשכל פילוסוף מחזיק מקל אכילה אחד
- משימה: תכנון פרוטוקול שִׁמְנֵעַ קפאון
- פתרון מספק: לפחות אחד יוכל לאכול ולחשוב אינסוף פעמים
- פתרון עדיף: כל הפילוסופים יוכלו לאכול ולחשוב אינסוף פעמים



פתרון טבעי: מערכת מעברים לפילוסוף והמקל ה- i

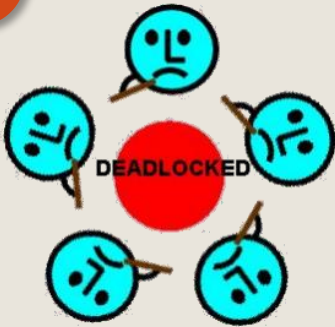


סנכרון בזוגות על כל פעולה בעלת שם זהה:

$$|||_{i=0}^{n-1} (Phil_i ||| Stick_i)$$



הפילוסופים נתקעים!



- אם לא קובעים פרוטוקול אפשר להגיע לקפאון
- לדוגמה: אם כל הפילוסופים מרימים את המקל השמאלי

מצב התחלתי: $\langle think, avail, think, avail, think, avail, think, avail, think, avail \rangle$

רצף פעולות: $request_{4,3}, request_{3,2}, request_{2,1}, request_{1,0}, request_{0,4}$

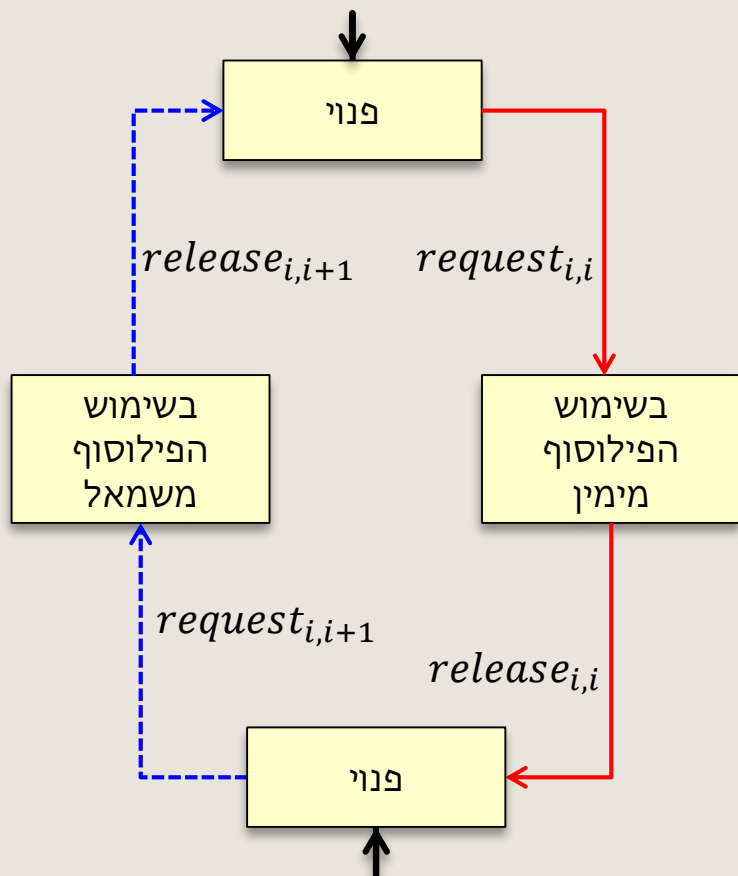
מצב אחרון: $\langle wait_R, occ_R, wait_R, occ_R, wait_R, occ_R, wait_R, occ_R, wait_R \rangle$

המצב האחרון הוא קפאון - כל פילוסוף ממתין לפינוי המקל מימינו

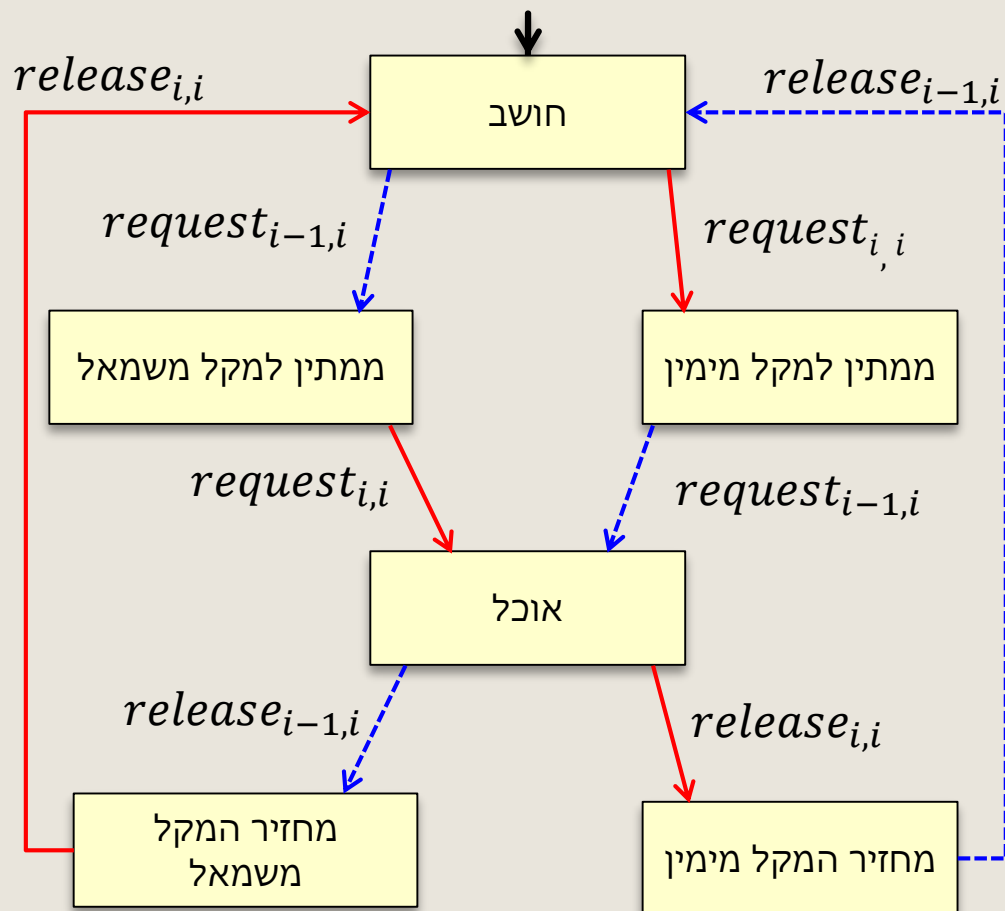


פתרון לבעיית הפילוסופים שננעלו

מקלות 1,3 מתחילים כאן



מקלות 0,2,4 מתחילים כאן



מערכות המעברים של הפילוסופים נשארות ללא שינוי

אפשר לבדוק באמצעות הכלי שפיתחם שהפרוטוקול עונה
על בעיית המניעה ההדדית ועל בעיית ההרעבה



חסינות לשגיאות (robustness)

- דרישה נוספת, מעבר למניעת קפאון, שלפעמים דורשים ממערכות מבזרות
 - במקרה של הפילוסופים הסועדים: הבטחת תכונות המניעה ההדדית וחוסר הרעבה גם אם אחד הסועדים "תקול"
 - פילוסוף "תקול" נתקע במצב חשיבה לעד
 - אפשר להוסיף חסינות ע"י:
- הפילוסוף ה- $i+1$ יכול לקחת את המקל ה- $i+1$ גם אם הפילוסוף i חושב
- מוסיפים משתנה בוליאני x_i המסמן לסועדים האחרים אם הפילוסוף i חושב

כדי להכניס את המשתנים,
עוברים ממודל של מערכות מעברים למודל של מערכת ערוצים

אלגוריתם בדיקה

איך נבדוק שאין הרעבה

במערכת ערוצים $[PG_1 | \dots | PG_n]$

או בקוד פרומלה?



תוכן

קפאון



הגדרת המושג "תכונת זמן ליניארי" (Linear Time Property)



שקילות עקבות (trace equivalence) ותכונות זמן ליניארי



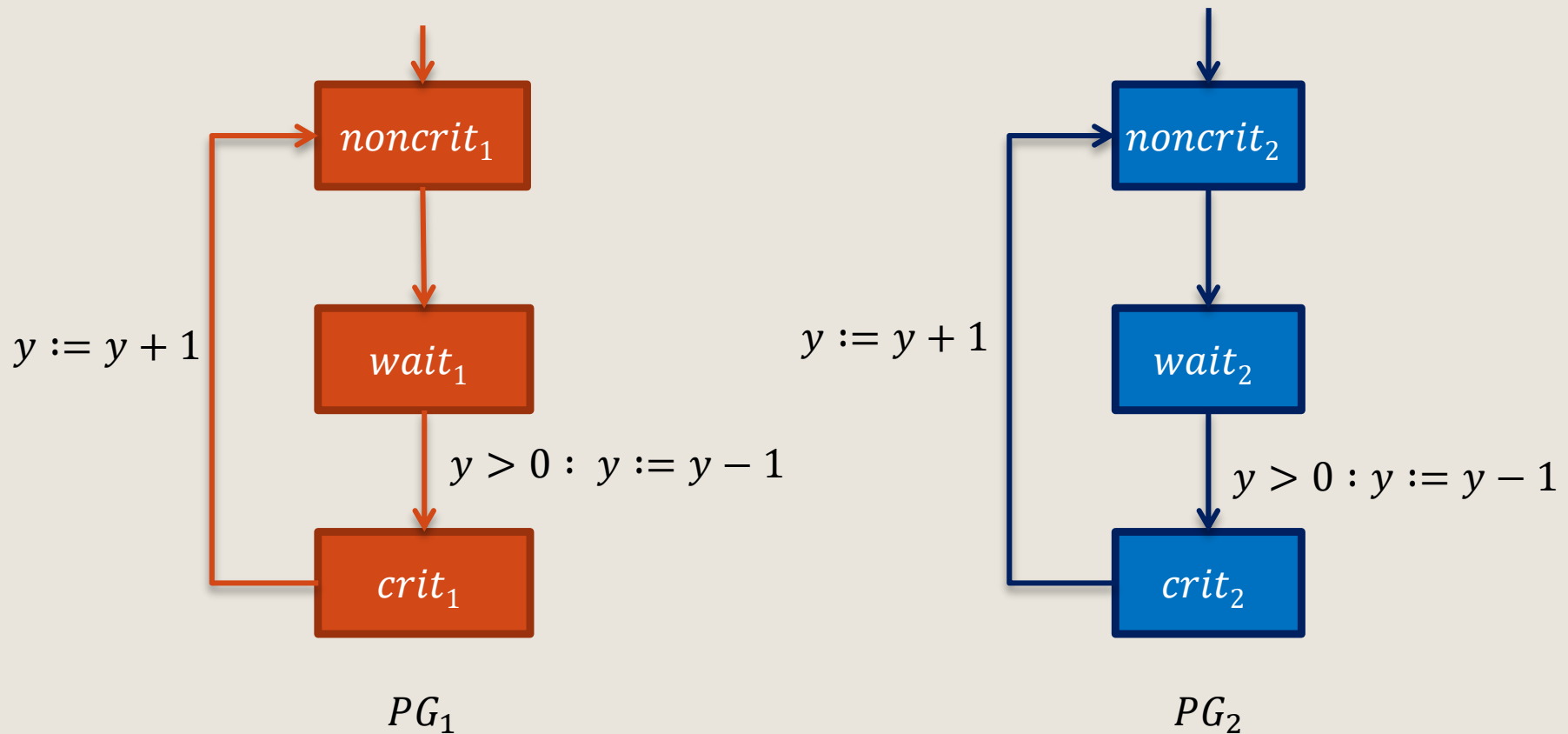
שמורות (invariants)





דוגמה רצה:

מניעה הדדית מבוססת אָתֶת (semaphore)

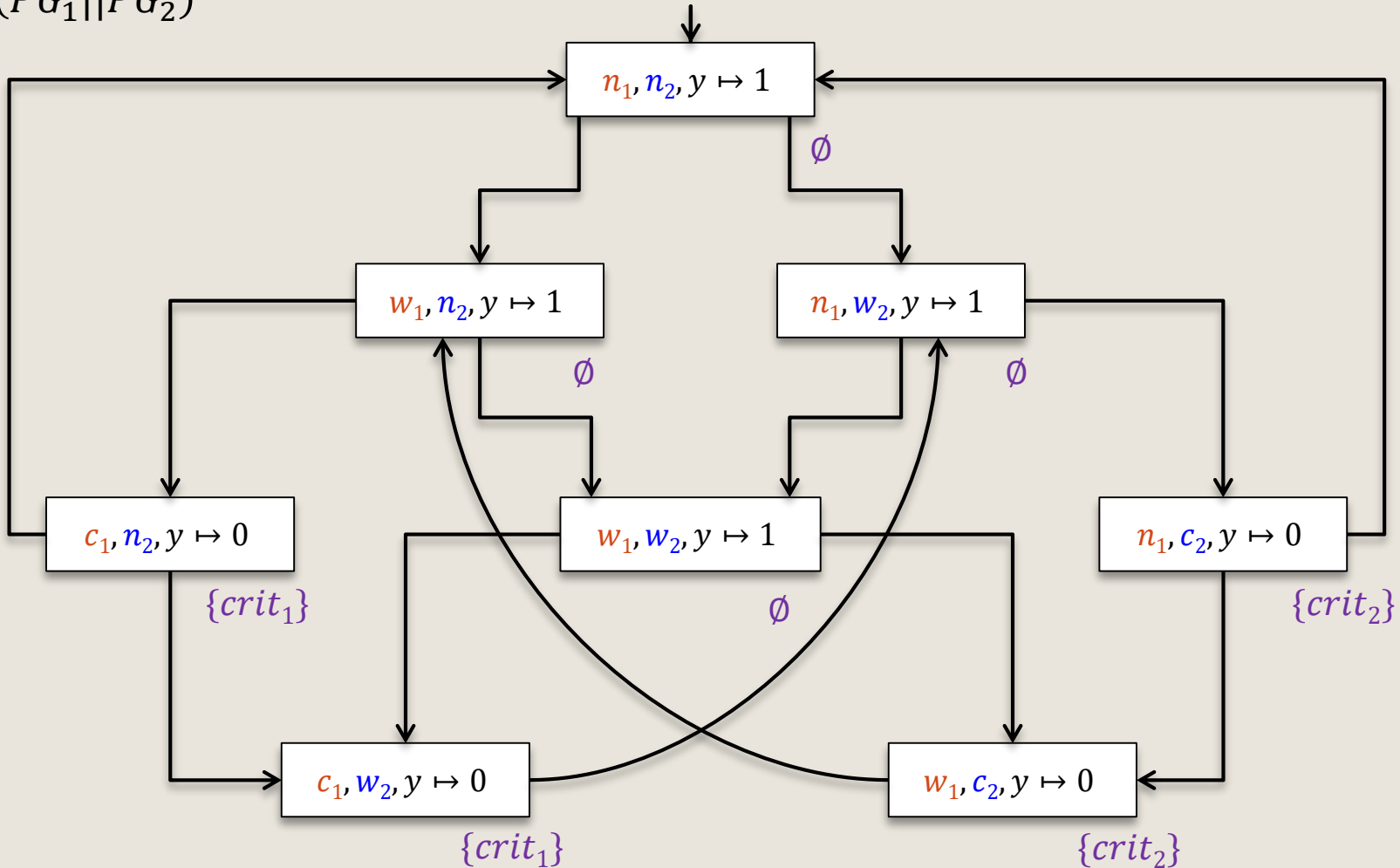


$y = 0$ מסמל "המנעול בשימוש", $y = 1$ מסמל "המנעול חופשי"



האם השגנו מניעה הדדית?

$TS(PG_1 || PG_2)$



כן, מכיוון שאין מצב נגיש המכיל גם את $crit_1$ וגם את $crit_2$



מריצות לתכונות

ריצות

$$s_0 \xrightarrow{\alpha_1} s_1 \xrightarrow{\alpha_2} s_2 \xrightarrow{\alpha_3} s_3 \cdots$$



מסלולים

$$s_0 s_1 s_2 s_3 \cdots$$



רצפי עקבות

$$L(s_0)L(s_1)L(s_2)L(s_3)\cdots$$

האם כל רצפי העקבות
מקיימים את התכונה?

כן



תכונה

קבוצה של מילים מעל הא"ב 2^{AP}



סימונים לטיפול במקטעי מסלול

עבור מקטע מסלול $\pi = s_0 s_1 \dots$

$$\text{first}(\pi) = s_0$$

• אם π סופי, באורך n , נסמן $\text{last}(\pi) = s_n$

• עבור $j \geq 0$

• האות ה- j , s_j , של π תסומן $\pi[j]$

• הרישא $s_0 s_1 \dots s_j$ תסומן $\pi[..j]$

• הסיפא $s_j s_{j+1} \dots$ תסומן $\pi[j..]$



עקבות (traces)

- תהי $TS = \langle S, Act, \rightarrow, I, AP, L \rangle$ מערכת מעברים **בלי מצבים סופניים** (ללא קיפאון). זה אומר שכל המסלולים המקסימאליים הם אינסופיים.

- העקבות של מקטע מסלול $\pi = s_0 s_1 \dots$

$$trace(\pi) = L(s_0)L(s_1) \dots$$

- קבוצת העקבות של קבוצת מסלולים Π :

$$trace(\Pi) = \{ trace(\pi) : \pi \in \Pi \}$$

- סימונים:

$$Traces_{fin}(s) = trace(Paths_{fin}(s))$$

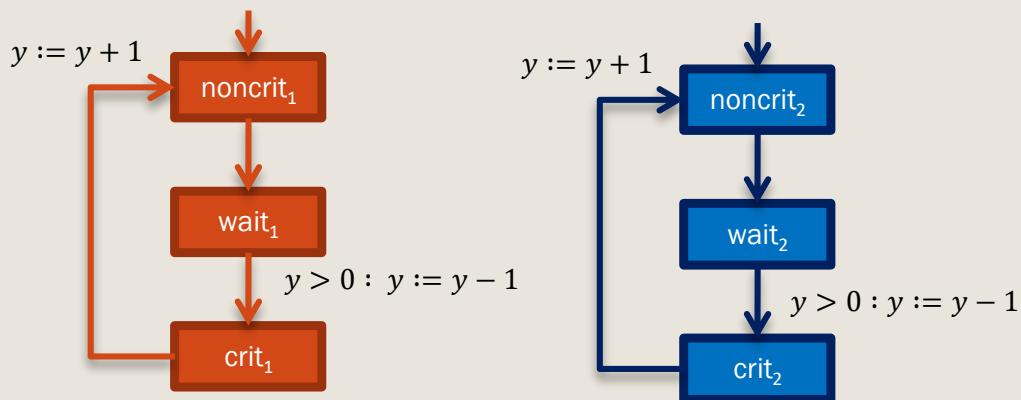
$$Traces(s) = trace(Paths(s))$$

$$Traces_{fin}(TS) = \bigcup_{s \in I} Traces_{fin}(s)$$

$$Traces(TS) = \bigcup_{s \in I} Traces(s)$$



דוגמה: מסלול ועקבותיו



- נניח $AP = \{crit_1, crit_2\}$
- מצב מתויג ב- $crit_i$ אם יש תהליך העומד במקום $crit_i$

- דוגמה למסלול:

$$\pi = \langle n_1, n_2, y = 1 \rangle \rightarrow \langle w_1, n_2, y = 1 \rangle \rightarrow \langle c_1, n_2, y = 0 \rangle \rightarrow \langle n_1, n_2, y = 1 \rangle \rightarrow \langle n_1, w_2, y = 1 \rangle \rightarrow \langle n_1, c_2, y = 0 \rangle \rightarrow \dots$$

- עקבות המסלול:

$$trace(\pi) = \emptyset \emptyset \{crit_1\} \emptyset \emptyset \{crit_2\} \emptyset \emptyset \{crit_1\} \emptyset \emptyset \{crit_2\}$$



תכונות זמן ליניארי

Linear Time Properties

הגדרה: תכונת זמן-ליניארי P היא תת קבוצה של המילים האינסופיות מעל האלף-בית 2^{AP} ז"א, $P \subseteq (2^{AP})^\omega$

אם מערכת מעברים TS מקיימת תכונת זמן ליניארי P נכתוב $TS \models P$

הגדרה: $TS \models P$ אם ורק אם $Traces(TS) \subseteq P$

במילים: מערכת מעברים TS מקיימת תכונת זמן ליניארי P אם ורק אם כל רצפי העקבות האינסופיים שהתוכנית ייצר מקיימים את התכונה



הגדרה: תכונת זמן לינארי היא תת-קבוצה $P \subseteq (2^{AP})^\omega$

הגדרה: $Traces(TS) \subseteq P$ אם ורק אם $TS \models P$

דוגמה

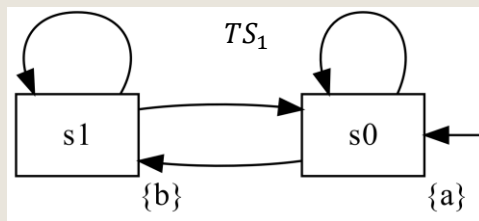
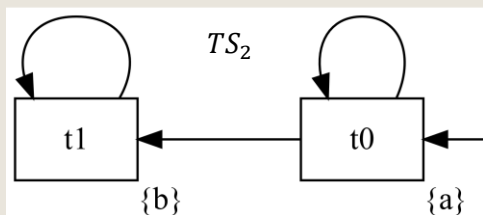
טענה: התכונה "לכל מצב במערכת יש עוקב שתיוגו $\{a\}$ " איננה תכונת זמן לינארי

הוכחה:

1. נניח, לקראת שלילה, שקיימת P המתארת את התכונה כפי שהוגדר למעלה.

2. ז"א: $Traces(TS) \subseteq P$ אם ורק אם לכל מצב של TS יש עוקב שתיוגו $\{a\}$

3. נבחן שתי מערכות מעברים:



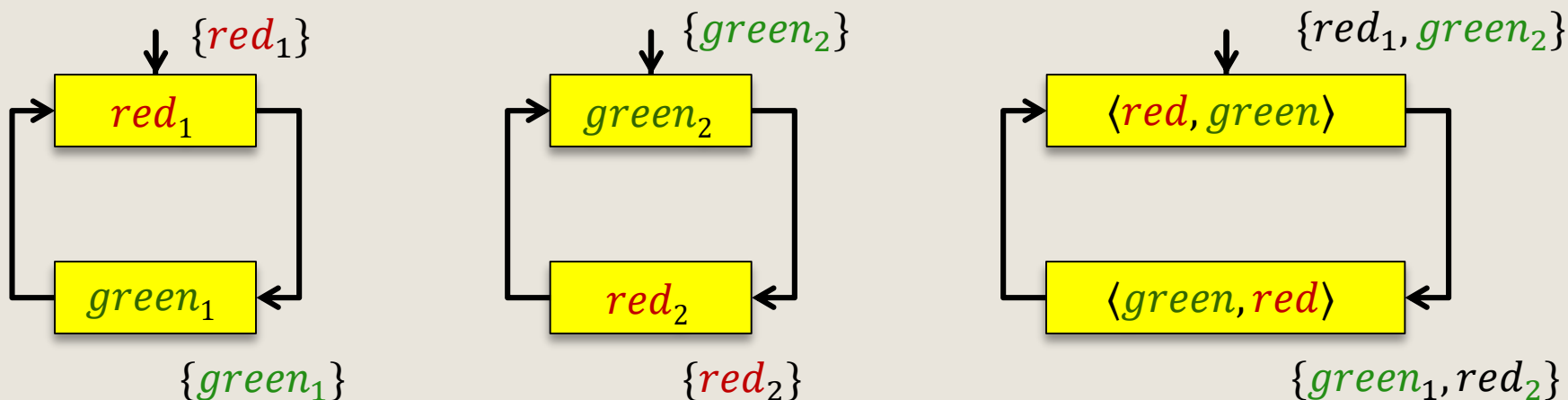
4. על פי 2, מכיוון שב- TS_1 לכל מצב יש עוקב שתיוגו $\{a\}$, $Traces(TS_1) \subseteq P$

5. בגלל ש- $Traces(TS_2) \subseteq Traces(TS_1)$ מקבלים גם: $Traces(TS_2) \subseteq P$

6. בסתירה ל-2 (כי ל- $t1$ אין עוקב שתיוגו $\{a\}$).



דוגמה: רמזורים מסונכרנים



"הרמזור הראשון יהיה ירוק לעיתים תכופות (אינסוף פעמים)"

$$P = \{A_1 A_2 \dots \in (2^{AP})^\omega : \forall j \in \mathbb{N}, \exists i > j . green_1 \in A_i\}$$

כל המילים האינסופיות מהצורה $A_1 A_2 A_3 \dots$ מעל 2^{AP} כך ש $green_1 \in A_i$ עבור אינסוף i -ים.
למשל, P מכילה את המילים:

$\{red_1, green_2\} \{green_1, red_2\} \{red_1, green_2\} \{green_1, red_2\} \dots$

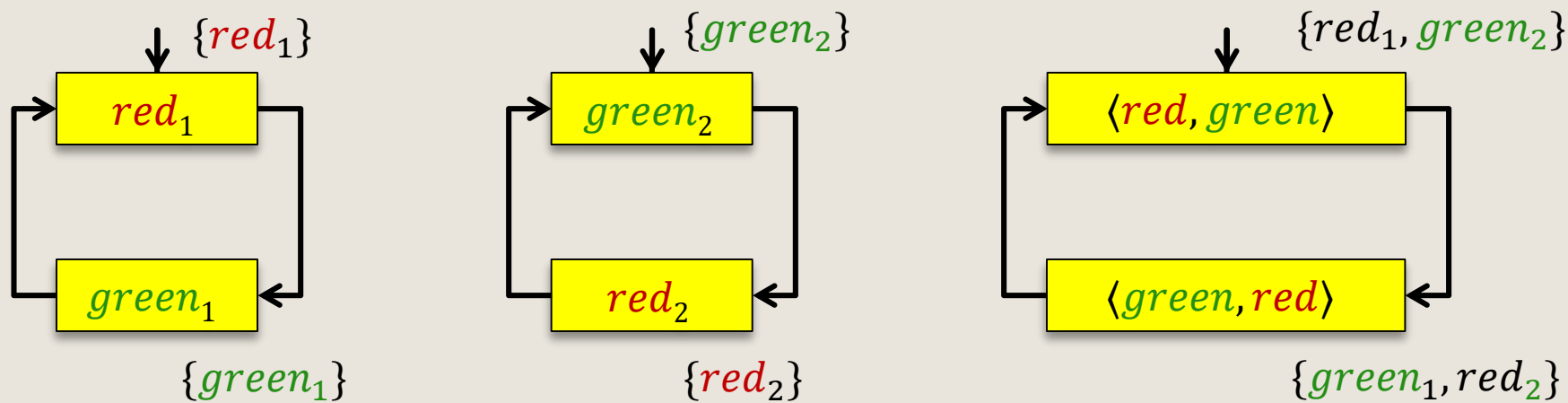
$\emptyset \{green_1\} \emptyset \{green_1\} \emptyset \{green_1\} \emptyset \{green_1\} \dots$

$\{red_1, green_1\} \{red_1, green_1\} \{red_1, green_1\} \dots$

$\{green_1, green_2\} \{green_1, green_2\} \{green_1, green_2\} \dots$



דוגמה: רמזורים מסונכרנים



P = "אף פעם לא מדליקים אור ירוק בשני הרמזורים ביחד"

$$P = \{A_1 A_2 \dots \in (2^{AP})^\omega : \forall i \in \mathbb{N}. green_1 \notin A_i \vee green_2 \notin A_i\}$$

כל המילים האינסופיות מהצורה $A_1 A_2 A_3 \dots$ מעל 2^{AP} כך שלכל i , $green_1 \notin A_i$ או $green_2 \notin A_i$

למשל, P מכילה את המילים:

$\{red_1, green_2\} \{green_1, red_2\} \{red_1, green_2\} \{green_1, red_2\} \dots$

$\emptyset \{green_1\} \emptyset \{green_1\} \emptyset \{green_1\} \emptyset \{green_1\} \dots$

$\{red_1, green_1\} \{red_1, green_1\} \{red_1, green_1\} \dots$



איך נגדיר מניעה הדדית?

• "תמיד נמצא לכל היותר אחד מהתהליכים בקטע הקריטי"

• נניח $AP = \{crit_1, crit_2\}$

• הפסוקים האטומיים האחרים אינם רלוונטיים לתכונה הזאת

• תאור כתכונת זמן ליניארי:

$$P_{\text{mutex}} = \{A_0 A_1 \dots : \{crit_1, crit_2\} \not\subseteq A_i \text{ for all } i\}$$

• דוגמאות למילים אינסופיות ב- P_{mutex} :

• $(\{crit_1\}\{crit_2\})^\omega$

• $(\{crit_1\}\{crit_1\})^\omega$

• דוגמאות למילים אינסופיות שאינן ב- P_{mutex} :

• $(\{crit_1\}\emptyset\{crit_1, crit_2\})^\omega$

• $(\{crit_1, crit_2\}\emptyset\{crit_1, crit_2\}\emptyset)^\omega$



Gary L. Peterson



איך נגדיר מניעת הרעבה?

• "תהליך הרוצה להיכנס לקטע הקריטי ייכנס בסופו של דבר"

• נגדיר $AP = \{wait_1, crit_1, wait_2, crit_2\}$

• כתיבה כתכונת זמן ליניארי:

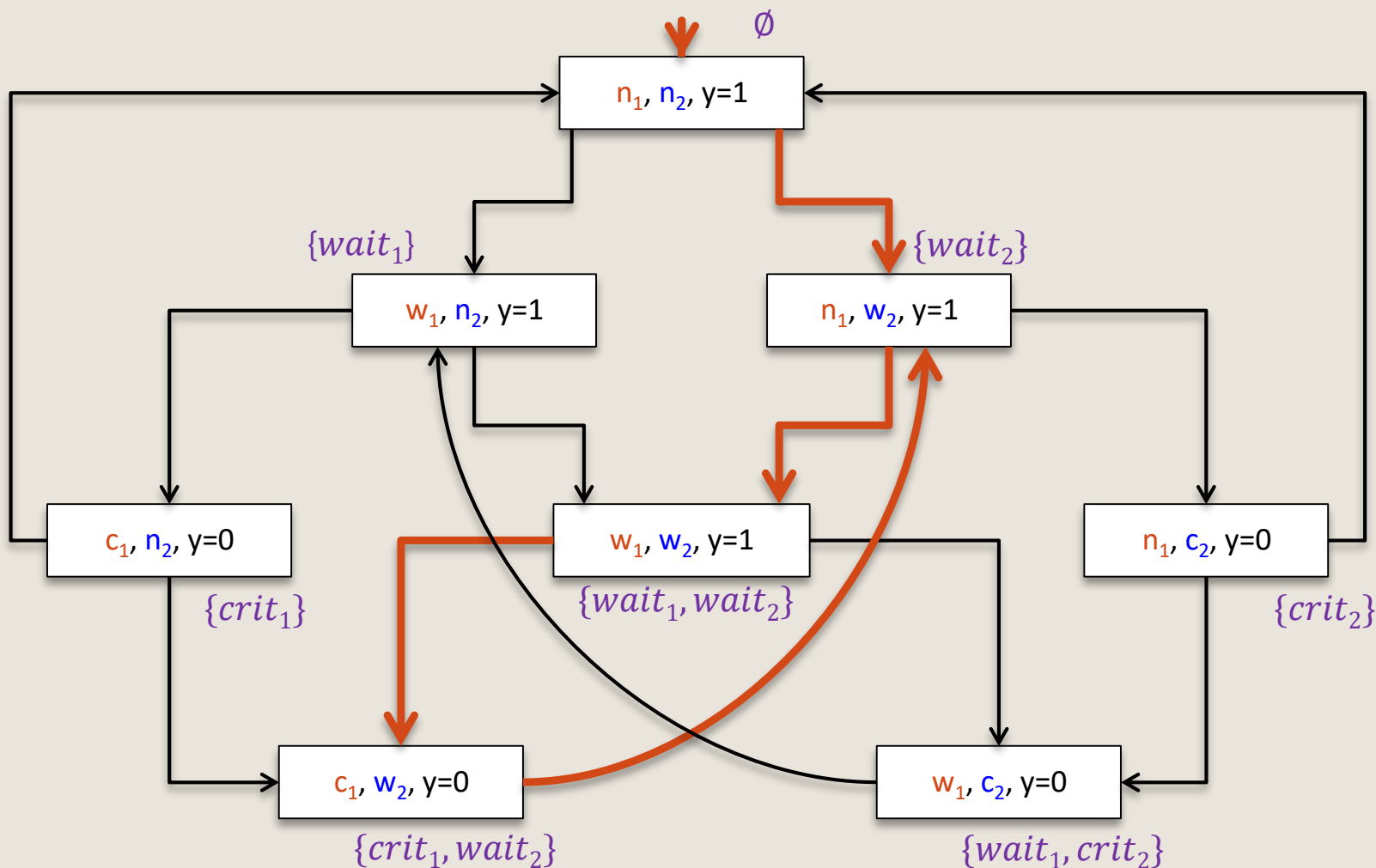
$$P_{\text{nostarve}} = \left\{ A_0 A_1 \dots : \left(\bigwedge_{j \in \mathbb{N}} j . wait_1 \in A_j \right) \Rightarrow \left(\bigwedge_{j \in \mathbb{N}} j . crit_1 \in A_j \right) \right\} \\ \cup \\ \left\{ A_0 A_1 \dots : \left(\bigwedge_{j \in \mathbb{N}} j . wait_2 \in A_j \right) \Rightarrow \left(\bigwedge_{j \in \mathbb{N}} j . crit_2 \in A_j \right) \right\}$$

• סימון: "קיימים אינסוף אינדקסים כך ש..."

$$\left(\bigwedge_{j \in \mathbb{N}} j . wait_1 \in A_j \right) \equiv (\forall k > 0 . \exists j > k . wait_1 \in A_j)$$

האם האלגוריתם מקיים את התכונה P_{nostarve} ?

האם האלגוריתם מבטיח חוסר הרעבה?



$\emptyset (\{wait_2\}\{wait_1, wait_2\}\{crit_1, wait_2\})^\omega \in Traces(TS) \setminus P_{no starve}$:לא



עֵדוֹן דְּרִישׁוֹת וּשְׁקִילוֹת עֵקֶבּוֹת

- **כשבונים מערכת:**

מתחילים בהגדרת דרישות כלליות ומפרטים את הדרישות במהלך הפיתוח

- **מבחינה פורמאלית:**

מתחילים במודל המגדיר ריצות ומעדנים אותו ככל שמבינים יותר איך המערכת צריכה לפעול – עד שמגיעים למימוש

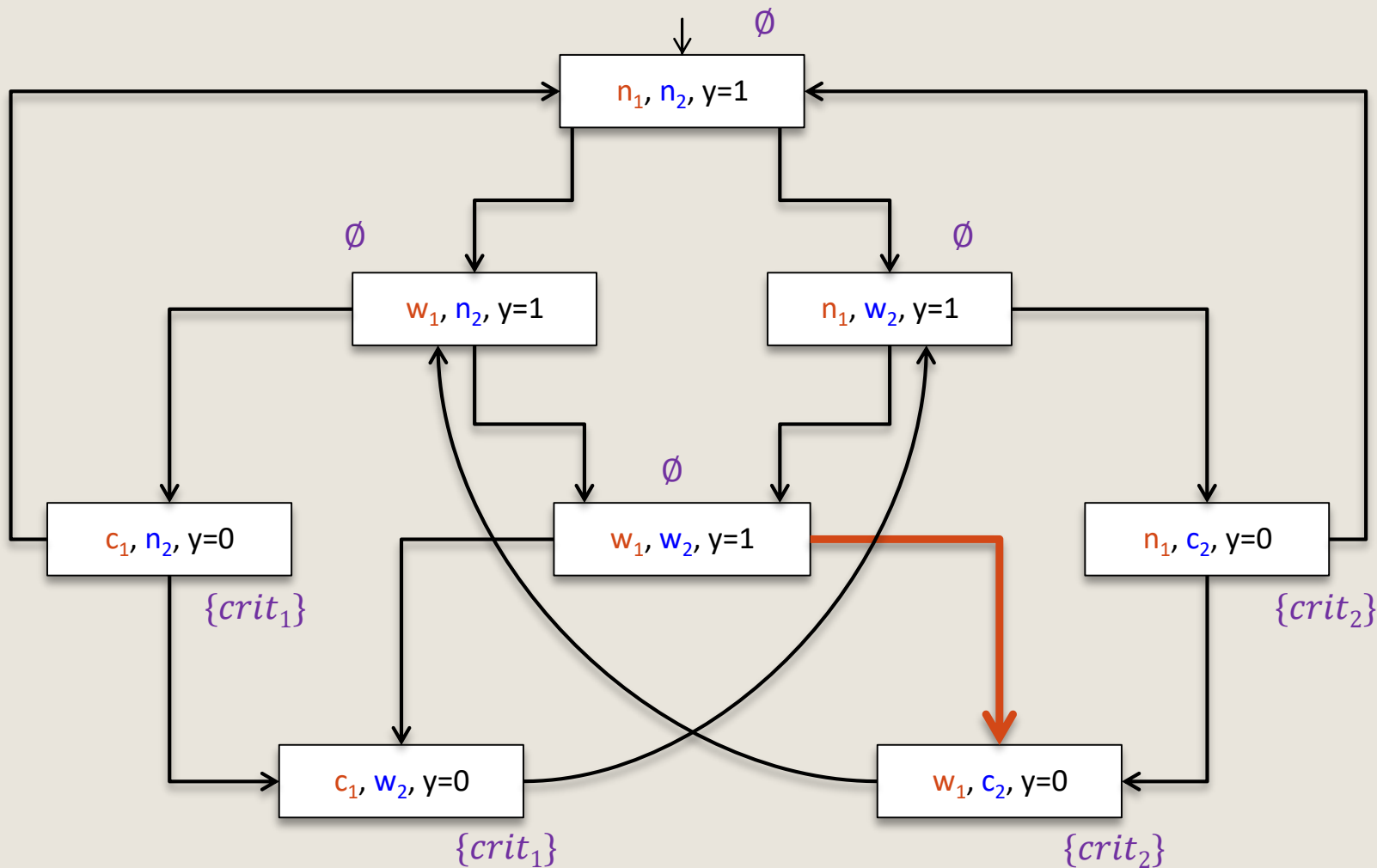
- מתמטית: $Traces(TS_L) \subseteq Traces(TS_H)$

מיתרונות הגישה הפורמאלית:

- אפשר לבדוק תכונות בכל שלב של הפיתוח
- אפשר לוודא שהמימוש הוא עֵדוֹן של הדרישות
- אפשר לוודא, מול הלקוח, כבר בשלב מוקדם, שהדרישות הובנו



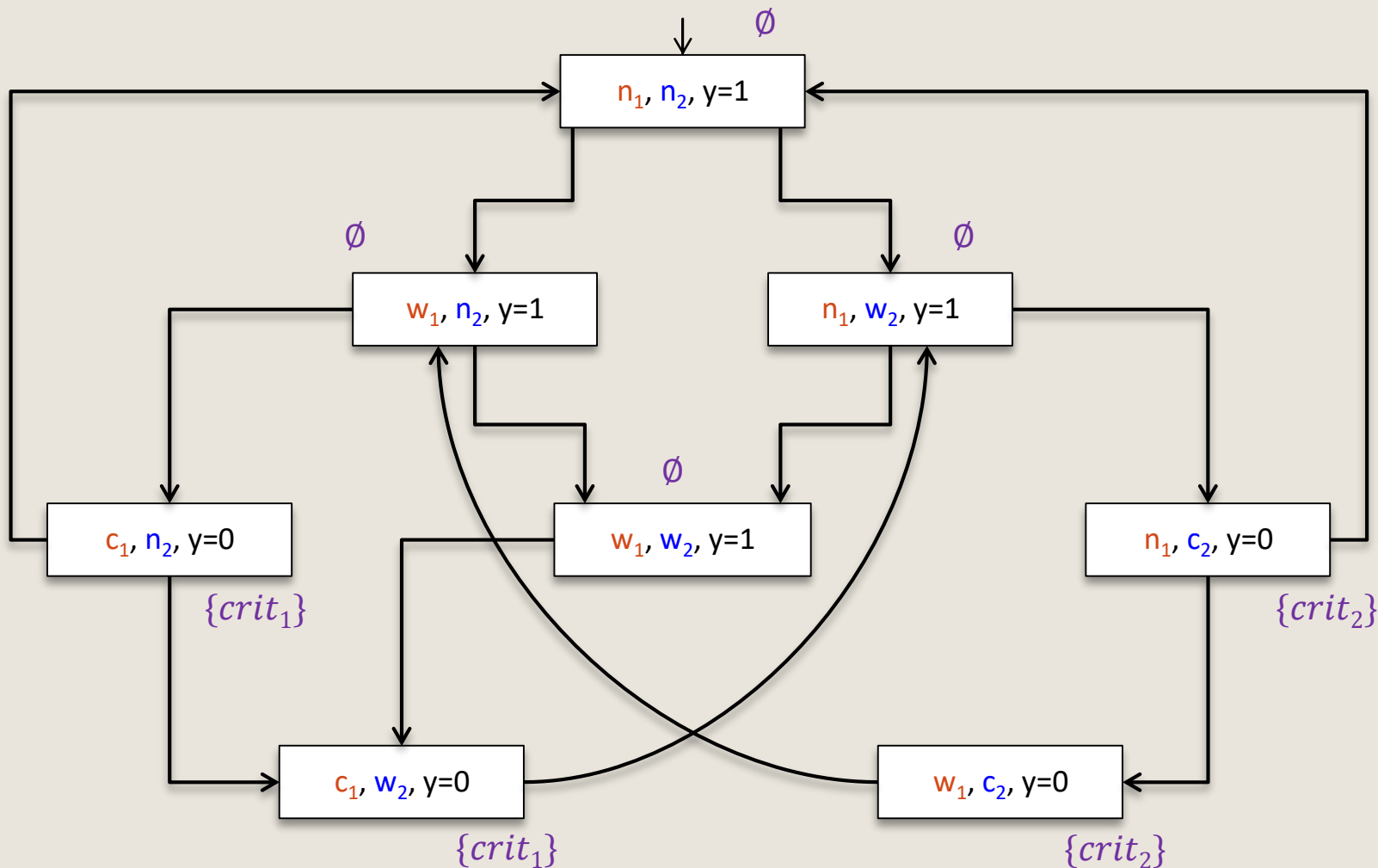
דוגמה: אלגוריתם מניעה הדדית



האלגוריתם מקיים את התכונה P_{mutex}



דוגמה: עידון אלגוריתם המניעה ההדדית



לא צריך לבדוק: גם הגרסה הזאת (הורדנו קשת) מקיימת את התכונה P_{mutex}



שקילות עקבות ותכונות זמן ליניארי

עבור מערכות מצבים TS ו TS' בלי מצבים סופניים:

$$Traces(TS) \subseteq Traces(TS')$$

אם ורק אם

לכל תכונת זמן ליניארי P מתקיים: אם $TS' \models P$ אז $TS \models P$

$$Traces(TS) = Traces(TS')$$

אם ורק אם

TS ו TS' מקיימות את אותן תכונות זמן ליניארי



שקילות עקבות ותכונות זמן ליניארי

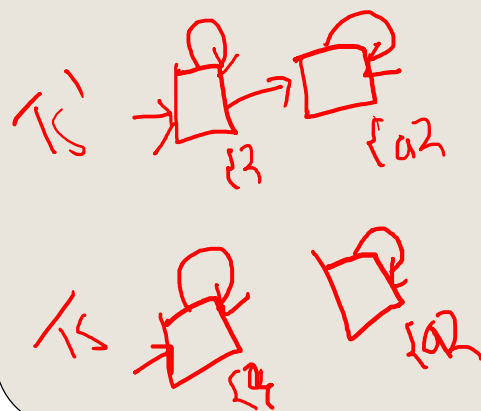
עבור מערכות מצבים TS ו TS' בלי מצבים סופניים:

$$Traces(TS) \subseteq Traces(TS')$$

אם ורק אם

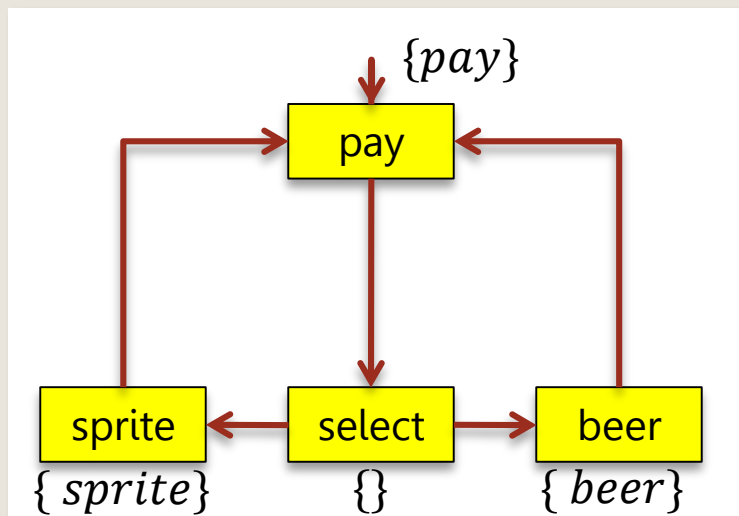
לכל תכונת זמן ליניארי P מתקיים: אם $TS' \models P$ אז $TS \models P$

"קיימת ריצה שמגיעה למצב המקיים את הפסוק האטומי a אינסוף פעמים".

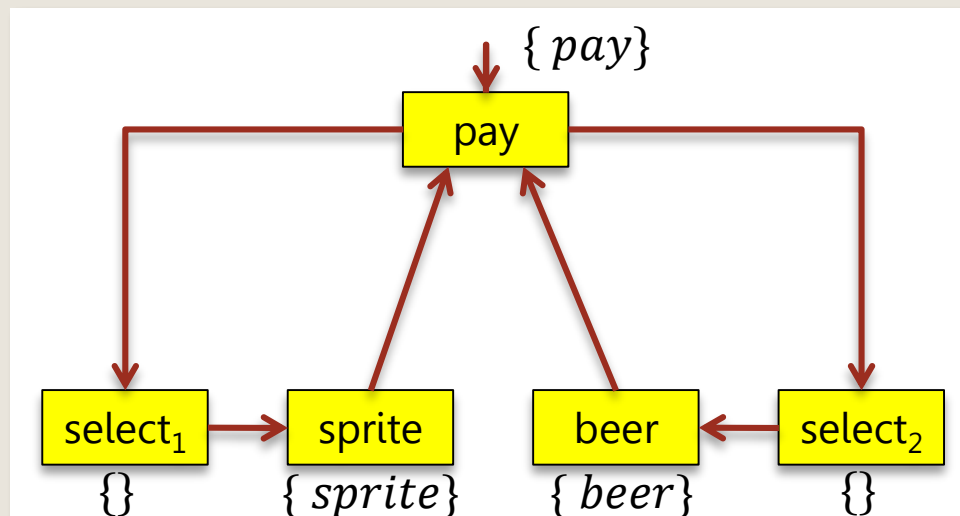


לצורך: בואו אנוכי תכנת זמן-ליניארי
נניח בואו, שקימת תכנת זמן-ליניארי מופיעה P
כל a ב TS אז מקימת את האות a

דוגמה: שתי מכונות שתייה



$\equiv$



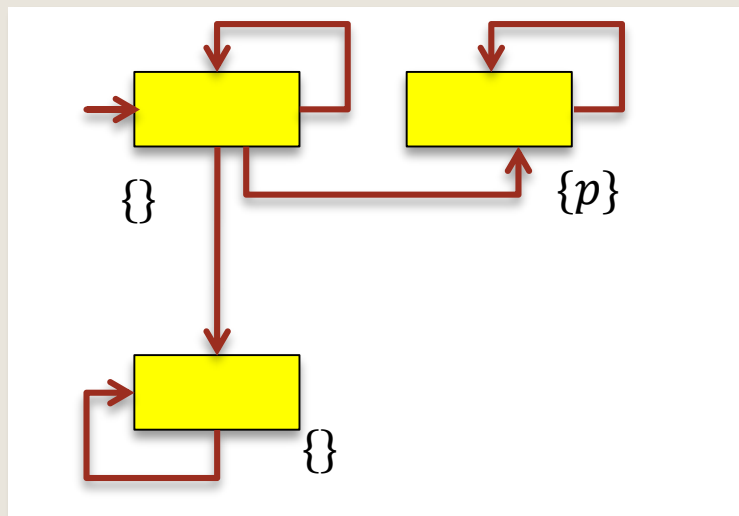
$$AP = \{pay, sprite, beer\}$$

אין תכונת זמן ליניארי שיכולה להבדיל בין שתי המכונות האלה

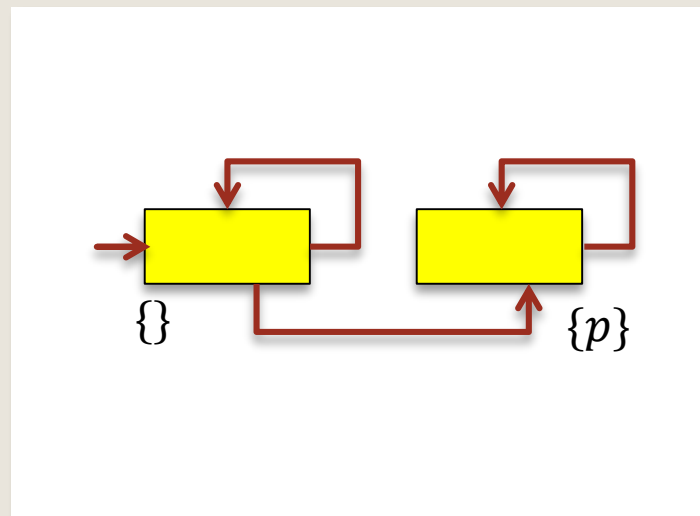
מסקנה: התכונה "יש ל- TS ארבעה מצבים" איננה תכונת זמן לינארי.



דוגמה: "הכל צפוי והרשות נתונה" (אבות ג' טו')



≡



אין תכונת זמן ליניארי שיכולה להבדיל בין שתי המכונות האלה

מסקנה: התכונה "מכל מצב יש מסלול למצב המקיים את התכונה p "
איננה תכונת זמן ליניארי.



סוגים של תכונות



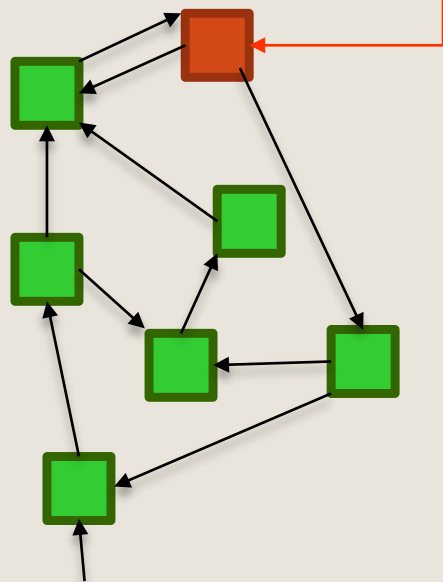


הגדרה: שמורות (invariants)

- תכונת זמן ליניארי P_{inv} היא **שמורה** אם קיים תנאי ϕ כך ש-

$$P_{inv} = \{ A_0 A_1 \dots : \forall j . A_j \models \phi \} \subseteq (2^{A^P})^\omega$$

מספיק שיהיה מצב נגיש
אחד שלא מקיים את ϕ
כדי לקבוע $TS \neq P_{inv}$



- ϕ נקרא תנאי השמורה (invariant condition)

- **קל להוכיח:** $TS \models P_{inv}$ אם ורק אם

$$trace(\pi) \in P_{inv} \text{ לכל מסלול } \pi \text{ של } TS$$

$$L(s) \models \phi \text{ לכל מצב } s \text{ השייך למסלול של } TS$$

$$L(s) \models \phi \text{ לכל מצב } s \in Reach(TS)$$

- הוכחת קיום תכונת שמורה באינדוקציה:

- ϕ חייב להתקיים בכל מצב התחלתי

- נכונות ϕ נשמרת תחת כל מעבר (מספיק בתחום הנגיש)



דוגמאות

מי מהתכונות הבאות הן שמורה? הוכיחו טענותיכם.

$$P_1 = \{\sigma \in (2^{AP})^\omega : \forall i. \sigma[i] \models p \rightarrow (q \vee \neg r)\}$$

$$P_2 = \{\sigma \in (2^{AP})^\omega : \sigma[0] \models (p \rightarrow q) \wedge \forall i. (p \in \sigma[i] \vee q \notin \sigma[i]) \rightarrow \sigma[i+1] \models (p \vee \neg q)\}$$

$$P_3 = \{\sigma \in (2^{AP})^\omega : p \in \sigma[0] \rightarrow \forall i. p \in \sigma[i]\}$$

איך נוכיח שתכונה איננה שמורה?



דוגמה

קבוצת המצבים:

$$S = \{0,1\}^*$$

יחס המעברים:

$$\{w_1 0 w_2 1 w_3 \rightarrow 1 w_1 w_2 w_3 : w_1, w_2, w_3 \in \{0,1\}^*\}$$

$$\cup$$

$$\{w_1 1 w_2 0 w_3 \rightarrow 1 w_1 w_2 w_3 : w_1, w_2, w_3 \in \{0,1\}^*\}$$

$$\cup$$

$$\{w_1 0 w_2 0 w_3 \rightarrow 0 w_1 w_2 w_3 : w_1, w_2, w_3 \in \{0,1\}^*\}$$

$$\cup$$

$$\{w_1 1 w_2 1 w_3 \rightarrow 0 w_1 w_2 w_3 : w_1, w_2, w_3 \in \{0,1\}^*\}$$

חידה: לאיזה מצב סופי נגיע אם נתחיל במצב נתון?

תשובה: זוגיות מספר האחדות היא שמורה



בדיקת שמורות

- בדיקה עבור פסוק ϕ = האם התכונה מתקיימת בכל מצב נגיש
 - שימוש בגרסה של אלגוריתם סריקת גרף (DFS או BFS)
 - בהנחה שמערכת המעברים סופית
- בדרך כלל, עדיף חיפוש DFS על BFS
 - רוב הפאגים המעניינים קורים לאחר רצף ארוך יחסית של פעולות
- ביצוע חיפוש DFS קדימה
 - אם מצאנו מצב s כך ש ϕ $\neq s$ מסיקים ש ϕ אינו שמורה
- אפשרות אחרת: חיפוש אחורה
 - מתחילים מהמצבים בהם ϕ אינה מתקיימת ($s \neq \phi$)
 - מחשבים את המצבים הקודמים $Pre^*(s)$ באמצעות DFS או BFS
 - אם הגענו למצב התחלתי ($I \cap Pre^*(s) \neq \emptyset$) מסיקים ש ϕ אינה שמורה



בדיקת שמורה באמצעות DFS

קלט: מערכת מעברים **סופית** TS ונוסחה ϕ
פלט: **true** אם TS מקיימת את השמורה "תמיד ϕ ", אחרת **false**

```
set of states  $R := \emptyset$  (* קבוצת המצבים בהם ביקרנו *)
stack of states  $U := \epsilon$ ; (* מחסנית מצבים "לטיפול" *)
boolean  $b := \text{true}$ ; (* כל המצבים ב  $R$  מקיימים את  $\phi$  *)
for all  $s \in I$  do
    if  $s \notin R$  then
        visit( $s$ ) (* מתחילים DFS מכל מצב התחלתי *)
    fi
od
```



בדיקת שמורה באמצעות DFS

procedure visit(state s)

 pus(s , U);

$R := R \cup \{s\}$;

repeat

$s' := \text{top}(U)$;

if $\text{Post}(s') \subseteq R$ **then**

 pop(U);

$b := b \wedge (s' \models \phi)$;

else

take any $s'' \in \text{Post}(s') \setminus R$;

 pus(s'' , U);

$R := R \cup \{s''\}$;

fi

until $(U = \epsilon \vee \neg b)$

endproc

(* פרוצדורה לביקור במצב *)

(* דוחפים את המצב למחסנית *)

(* ומוסיפים אותו למצבים שכבר ביקרנו *)

(* בודקים אם s' מקיימת את ϕ *)

(* גילינו מצב נגיש חדש s'' *)



סיבוכיות זמן

- נניח שניתן לסרוק כל $s' \in Post(s)$ בזמן $\theta(|Post(s)|)$
- הנחה תקפה כאשר מייצגים את $Post(s)$ ע"י רשימות סמיכות
- סיבוכיות זמן בדיקת שמורה: $O(N \cdot |\phi| + M)$
- N מסמל את מספר המצבים הנגישים
- $M = \sum_{s \in S} |Post(s)|$ הוא מספר המעברים (בחלק הנגיש) של TS
- בדרך כלל לא מייצגים את רשימות הסמיכות באופן מפורש
- למשל: ניתן להשתמש מעבר ישיר על גרפי התוכנית. במקרה זה, $Post(s)$ מתקבל מהכללים של יחס המעברים.



אלגוריתם שנותן גם דוגמה נגדית

set of states $R := \emptyset$

(* קבוצת המצבים בהם ביקרנו *)

stack of states $U := \epsilon$;

(* מחסנית מצבים "לטיפול" *)

boolean $b := \text{true}$;

(* כל המצבים ב R מקיימים את ϕ *)

while $((I \setminus R \neq \emptyset) \wedge b)$ **do**

take any $s \in I \setminus R$

 visit(s)

od

if b **then**

 return “yes”

else

 return “no”, **reverse**(U)

fi



דוגמה נגדית שימושית יותר מאשר הוכחת נכונות

כי דוגמה נגדית יכולה להצביע על באג אמיתי

